

Benchmarks

CSC322 Group E — CalcEngine

The brief sets two hard numbers:

A single cell edit in a workbook of 100,000 cells with a chain of 500 dependent cells must propagate within 50 milliseconds. A full recalculation of the same workbook must complete within 2 seconds.

Both are met with roughly two orders of magnitude in hand.

1. Running them

```
# The instrument. Release matters: a Debug build measures the absence of the optimiser.
dotnet run -c Release --project benchmarks/CalcEngine.Benchmarks

# Machine-readable, for tracking across commits.
dotnet run -c Release --project benchmarks/CalcEngine.Benchmarks -- --json benchmarks/
results/latest.json

# Fewer iterations, for a quick check during development.
dotnet run -c Release --project benchmarks/CalcEngine.Benchmarks -- --quick
```

The harness exits non-zero if any published target is missed, so it can be wired into CI as a regression gate.

The same two targets are also asserted by the ordinary test suite, in `tests/CalcEngine.Core.Tests/Performance/PerformanceTargetTests.cs` — the harness is the instrument, the tests are the alarm:

```
dotnet test --filter "Category=Performance"      # the targets only
dotnet test --filter "Category!=Performance"     # everything else
```

Those run in the Debug configuration and still pass, which is the point: a failure there means something regressed structurally, not that the machine was busy.

2. The benchmark workbook

Exactly 100,000 cells, containing exactly the 500-cell chain the brief describes, so the numbers answer the question that was asked rather than an easier one.

Sheet	Contents	Cells
Data	10,000 rows × 8 columns of numeric literals	80,000

Sheet	Contents	Cells
Calc	9,750 rows × 2 formulas: <code>=SUM(Data!Ar:Hr)</code> and <code>=ROUND(Ar*0.3,2)</code>	19,500
Chain	<code>=Data!A1*2</code> , then 499 cells each reading the one above	500
	Total	100,000

Editing `Data!A1` therefore reaches the whole 500-cell chain plus the two `Calc` cells for row 1 — 502 evaluations — and touches nothing else. Editing `Data!H10000` reaches nothing at all, because no formula reads that row. Those two facts are asserted, not assumed: the harness fails if the chain does not recompute.

3. Results

Measured on the development machine: .NET 8.0.30 on Linux x64, 4 logical processors, Release build, workstation GC. A target is treated as met only when the **worst** observed run is inside the budget — reporting a median that fits while the tail does not would be a way of not answering the question.

Benchmark	Target	Runs	Min	Median	p95	Max	Verdict
Load 100,000 cells (parse + first evaluation)	no published target	1	1214.76 ms	1214.76 ms	1214.76 ms	1214.76 ms	—
Full recalculation of 100,000 cells	< 2,000 ms	5	74.39 ms	75.04 ms	79.37 ms	79.37 ms	PASS
Single edit propagating through 500 dependents	< 50 ms	30	0.17 ms	0.21 ms	0.32 ms	0.32 ms	PASS
Single edit with no dependents	no published target	30	0.00 ms	0.00 ms	0.02 ms	0.08 ms	—
Edit inside a SUM range (range index lookup)	no published target	30	0.00 ms	0.01 ms	0.06 ms	0.10 ms	—
Find across 19,500 formula cells	no published target	10	16.08 ms	19.54 ms	44.29 ms	44.29 ms	—
Replace across 9,750 formulas, twice, with re-parse validation	no published target	5	483.37 ms	496.99 ms	536.08 ms	536.08 ms	—
Duplicate scan of 10,000 rows × 8 columns	no published target	10	42.03 ms	47.81 ms	64.61 ms	64.61 ms	—
Edit, undo and redo through the 500-cell chain	no published target	30	0.44 ms	0.48 ms	0.61 ms	0.65 ms	—

Headroom: 156× on the propagation target, 25× on the full recalculation.

4. Why the numbers look like that

The edit is 0.2 ms because the cost follows the affected subgraph, not the workbook. Three decisions do the work:

1. *Dense integer cell identifiers.* Adjacency is array indexing, not hashing.
2. *Generation-stamped visit marks.* The traversal never clears its mark arrays. Clearing 100,000 flags per keystroke would, on its own, cost more than the entire propagation does now.
3. *A spatial range index instead of expanded edges.* `=SUM(Data!A1:H1)` is one index entry, not eight edges, and it still fires when a previously empty cell inside the range is filled in.

Full recalculation is ~75 ms for 19,500 formula evaluations — about 4 µs per formula, most of it the `SUM` over eight cells. Parsing is not in this number: trees are built once when content is set and kept, which is also why loading is the slowest line in the table.

Loading is ~1.2 s for 100,000 edits, of which 19,500 are parsed. That is about 60 µs per formula parsed and is dominated by ANTLR. There is no published target for load, but it is the obvious next thing to attack — see §5.

Replace-all is ~250 ms per pass over 9,750 formulas because every rewritten formula is re-parsed to prove it still compiles before it is written. That check is the feature, not an overhead to remove: it is what turns "the replacement broke 400 formulas" into a list of cells that were left alone.

The duplicate scan is ~45 ms for 10,000 eight-column records — one linear pass building 10,000 keys, against the 50 million pairwise comparisons a naive implementation would perform.

5. What I would attack next, and why I did not

- **Parse throughput.** 60 µs per formula is ANTLR's `AdaptivePredict` doing full-context prediction on a small grammar. Setting the parser to `PredictionMode.SLL` with a fallback to LL on failure typically wins 2–5× and is a five-line change. I left it out because load time has no published target and the change deserves its own tests.
- **A formula-tree cache.** The 9,750 `Calc` formulas differ only in a row number; a workbook that interned structurally identical trees would parse a few and share them. Real spreadsheets do this. It is a genuine optimisation and a genuine source of aliasing bugs, so it wants more than a week.
- **Parallel evaluation of independent topological levels.** The order already identifies cells that cannot affect one another. With a 25× margin on the only published target, adding threads to a data structure this mutable would be buying risk I do not need.

6. Reproducing the environment

```
$ dotnet --version
8.0.130
$ nproc
4
```

Numbers from a different machine will differ; the shape should not. If the propagation figure ever approaches the same order of magnitude as the recalculation figure, the dependency graph has stopped restricting work to the affected subgraph, and that is the thing to go and look at.